

Lab 5: Interrupt-Driven Drone Mission and Safety Supervisor

ECE 3301 – Microcontrollers | Fall 2026

Instructor: Sasoun Torousian

Lab 5 : Interrupt-Driven Drone Mission and Safety Supervisor

1. Introduction

In Lab 4, you created a Drone Mission Timing and Safety Supervisor using a state machine and Timer0. Your controller could enter SAFE, ARMING, ARMED, and FAULT states, verify an ARM request for two seconds, detect communication loss, generate status indications, and immediately remove the ARMED condition when a fault was detected. There was one major limitation: the processor had to repeatedly check for important events. The main loop repeatedly checked the FAULT input, the COMM input, and the Timer0 overflow flag. This technique is called polling. This week you learned a different approach: interrupts.
An interrupt allows hardware to notify the processor that an event occurred. Instead of continuously asking whether something happened, the processor can continue executing the main program until hardware reports an important event. In this lab, you will redesign the Lab 4 drone supervisor using an interrupt-driven architecture. The goal is not simply to make a pushbutton generate an interrupt. You already know how to read a pushbutton. The goal is to redesign the architecture of the entire drone supervisor. Your new controller will use external interrupts for asynchronous drone events, a timer interrupt for system timing, interrupt priorities for events with different levels of urgency, event flags to communicate between interrupt routines and the main program, and a state machine to determine what the drone should do. By the end of this lab, your firmware will begin to resemble the structure used in larger real-time embedded systems.

2. What You Are Building

You will upgrade the Drone Mission Timing and Safety Supervisor from Lab 4. The system will still contain four operating states:

```
SAFE
ARMING
ARMED
FAULT
```

However, the way events reach the state machine will change. In Lab 4:

```
Hardware
   |
   v
Main Loop Polls Inputs
   |
   v
State Machine
   |
   v
Outputs
```

In this lab:

```
                PIC18F46K22
                     |
       +-------------+-------------+
       |             |             |
      INT0          INT1          INT2
      FAULT         ARM           COMM
       |             |             |
       +-------------+-------------+
                     |
                     v
                    ISR
                     |
                     v
                Event Flags
                     |
                     v
                 main()
                     |
                     v
               State Machine
                     |
                     v
             Drone Supervisor
```

A Timer2 interrupt will simultaneously maintain the system time:

```
Timer2
   |
   v
1 ms Interrupt
   |
   v
system_ms
```

```
    |
    +---- ARM qualification timing
    |
    +---- Communication timeout
    |
    +---- LED timing
    |
    +---- Button debounce timing
```

The important architectural rule is:

```
ISR = Detect and record the event

main() = Decide what the event means and perform the action
```

Do not place the complete drone-control behavior inside an ISR.

3. Learning Objectives

After completing this lab, you should be able to configure external interrupts on the PIC18F46K22, configure interrupt edge detection, configure Timer2 to generate a periodic interrupt, use multiple interrupt sources in the same application, use interrupt priorities, create `volatile` event flags shared between an ISR and the main program, keep an ISR short and predictable, implement non-blocking timing using a millisecond system clock, handle mechanical pushbutton bounce without placing delays inside an ISR, combine interrupts with a state machine, and explain when polling is still appropriate even in an interrupt-driven system.

4. Required Hardware

Use the same basic hardware architecture from Lab 4.

Quantity	Component	Purpose
1	Curiosity HPC with PIC18F46K22	Main controller
3	Pushbuttons	ARM, FAULT, COMM
4	LEDs	SAFE, ARMING, ARMED, FAULT
4	220 Ω to 470 Ω resistors	LED current limiting
As needed	Jumper wires	Connections

Quantity	Component	Purpose
1	Breadboard	External circuit

The pushbuttons are active LOW.

```
Button released = HIGH
Button pressed  = LOW
```

Therefore, pressing a button produces:

```
HIGH → LOW
```

which is a falling edge.

5. Input Assignment

Use the following interrupt-capable inputs.

Signal	PIC Pin	Interrupt	Purpose
FAULT	RB0	INT0	Critical drone fault
ARM	RB1	INT1	Operator ARM request
COMM	RB2	INT2	Communication heartbeat

This pin assignment is intentional. The PIC18F46K22 provides three dedicated external interrupt inputs, allowing this lab to demonstrate several asynchronous events without continuously polling each button.

6. Output Assignment

Use PORTD for the drone status indicators.

Signal	PIC Pin	Purpose
SAFE LED	RD0	Drone is safe/disarmed
ARMING LED	RD1	ARM qualification is occurring
ARMED LED	RD2	Propulsion permission

Signal	PIC Pin	Purpose
FAULT LED	RD3	Critical fault condition

At startup the system must enter SAFE.

```
State       = SAFE
SAFE LED    = ON
ARMING LED  = OFF
ARMED LED   = OFF
FAULT LED   = OFF
```

The ARMED output must never automatically become active after power-up or reset.

7. Part 1 – Configure the External Interrupts

Configure RB0, RB1, and RB2 as digital inputs. Remember that an interrupt requires more than configuring the pin. For an interrupt to execute:

```
EVENT
  |
  v
Interrupt Flag
  |
  v
Interrupt Enable
  |
  v
Global Enable
  |
  v
ISR
```

Configure all three external interrupts for falling-edge detection because the pushbuttons are active LOW. You will need to determine the appropriate registers and bits for:

```
INT0
INT1
INT2

INT0IF / INT0IE
INT1IF / INT1IE
INT2IF / INT2IE

INTEDG0
INTEDG1
INTEDG2
```

Before enabling an interrupt, clear its interrupt flag. Do not simply copy register settings without understanding them. During the demonstration you may be asked to explain why the interrupt is configured for a falling edge.

8. Part 2 – Interrupt Priorities

Not every event in a drone has the same urgency. Consider these three events:

```
Critical FAULT
ARM request
COMM heartbeat
```

A critical hardware fault is more important than an operator ARM request or a communication heartbeat. Enable interrupt priority mode. Configure the architecture so that:

```
FAULT / INT0 = HIGH priority

ARM / INT1   = LOW priority

COMM / INT2  = LOW priority
```

INT0 is special on the PIC18F46K22. When priority mode is enabled, INT0 is automatically a high-priority interrupt. Your firmware must therefore contain a high-priority ISR and a low-priority ISR. Conceptually:

```
              INTERRUPTS

         HIGH              LOW
          |                 |
       FAULT          ARM / COMM
          |                 |
          v                 v
      High ISR          Low ISR
```

If the processor is handling a lower-priority event and a critical FAULT occurs, the critical event must receive priority.

9. Part 3 – Event Flags

The interrupt routines must not execute the entire drone state machine. Create event variables similar to:

```
volatile uint8_t fault_event = 0;
volatile uint8_t arm_event   = 0;
volatile uint8_t comm_event  = 0;
```

The exact names are your choice. When the appropriate interrupt occurs, the ISR should record the event and clear the hardware interrupt flag. Conceptually:

```
FAULT edge
    |
    v
High ISR
    |
    +---- fault_event = 1
    |
    +---- Clear interrupt flag
    |
    v
Return to main()
```

Do not perform long calculations, LED sequences, communication processing, state-machine execution, or delays inside the ISR. The ISR should be short.

10. Part 4 – Timer2 Interrupt and the System Clock

Lab 4 used Timer0 to generate a 100-ms system tick, but the main program had to poll `TMR0IF`. You will now replace that architecture with a Timer2 interrupt. Configure Timer2 to generate an interrupt every:

```
1 ms
```

Assume:

```
FOSC = 16 MHz
FOSC / 4 = 4 MHz

Timer2 prescaler = 1:16
PR2 = 249
Timer2 postscaler = 1:1
```

Verify that these values produce approximately 1 ms. Create:

```
volatile uint32_t system_ms = 0;
```

Every Timer2 interrupt should increment:

```
system_ms++;
```

Your program now has a continuously running millisecond clock. You are no longer allowed to poll the Timer0 overflow flag as you did in Lab 4.

11. Part 5 – Preserve the Drone State Machine

Your controller must still implement:

```
SAFE
ARMING
ARMED
FAULT
```

You may use:

```
typedef enum
{
    SAFE,
    ARMING,
    ARMED,
    FAULT
} DroneState;
```

The state machine belongs in the main program, not inside an interrupt routine. The basic architecture should now look like:

```
Interrupts
    |
    v
Event Flags
    |
    v
```

```
while(1)
    |
    +---- Process critical events
    |
    +---- Process operator events
    |
    +---- Process communication events
    |
    +---- Execute state machine
    |
    +---- Execute timed tasks
    |
    +---- Update outputs
```

12. SAFE State

When the system is SAFE:

```
SAFE LED   = ON
ARMING LED = OFF
ARMED LED  = OFF
FAULT LED  = OFF
```

The controller remains SAFE until a valid ARM event occurs. An ARM interrupt does not mean that the drone should immediately become ARMED. It means:

```
"The operator requested arming."
```

The state machine must decide what happens next. When a valid ARM request occurs:

```
SAFE → ARMING
```

Record the starting time:

```
arming_start_time = system_ms;
```

13. ARMING State

The drone must remain in ARMING for:

```
2 seconds
```

Do not use:

```
__delay_ms(2000);
```

Instead use the system clock. Conceptually:

```
if((system_ms - arming_start_time) >= 2000)
{
    // Arming interval completed
}
```

During these two seconds, the processor must remain available to respond to other events. This is the major difference between blocking timing and real-time firmware.

14. Continuous ARM Requirement

There is an important engineering detail. The ARM interrupt tells you:

```
The ARM button was pressed.
```

It does not tell you:

```
The ARM button is still pressed.
```

Therefore, interrupts do not eliminate polling completely. During ARMING, you must verify the current ARM input level. If ARM is released before two seconds have elapsed:

```
ARMING → SAFE
```

If ARM remains pressed continuously for two seconds and no fault exists:

```
ARMING → ARMED
```

This demonstrates an important engineering principle:

```
Interrupt → detect an event

Polling → determine a current condition
```

Use whichever technique is appropriate for the information you need.

15. ARMED State

When ARMED:

```
SAFE LED   = OFF
ARMING LED = OFF
ARMED LED  = ON
FAULT LED  = OFF
```

Releasing ARM must return the system to SAFE.

```
ARMED + ARM released → SAFE
```

The ARM input may therefore still be polled while ARMED because the program needs to know its present condition, not merely whether a previous falling edge occurred.

16. Emergency FAULT Interrupt

FAULT is the most critical input in the system. A falling edge on RB0/INT0 must immediately generate the high-priority interrupt. The high-priority ISR should record:

```
fault_event = 1;
```

and clear the appropriate interrupt flag. The main program must process `fault_event` before normal state-machine behavior. A fault may occur while the controller is:

```
SAFE
ARMING
ARMED
```

The required transition is:

```
ANY STATE → FAULT
```

The ARMED output must be removed. Do not wait for the 1-ms timer, the two-second ARM interval, or the communication watchdog to finish.

17. FAULT State and Recovery

When the controller enters FAULT:

```
SAFE LED   = OFF
ARMING LED = OFF
ARMED LED  = OFF
FAULT LED  = ON
```

Releasing FAULT must not automatically return the drone to ARMED. Recovery is permitted only when:

```
FAULT input released
AND
ARM input released
```

Then:

```
FAULT → SAFE
```

The complete ARM procedure must be performed again before the drone can return to ARMED.

18. Communication Heartbeat Interrupt

In Lab 4, you manually detected the transition:

```
Released → Pressed
```

on the COMM button. That software edge-detection logic is no longer required. RB2/INT2 hardware will now detect the falling edge. When COMM is pressed:

```
HIGH → LOW
```

INT2 generates an interrupt. The low-priority ISR records:

```
comm_event = 1;
```

The main program processes the event. If the system is ARMED, a valid COMM event means:

```
"I recently heard from the communication system."
```

Record the time:

```
last_comm_time = system_ms;
```

19. Communication Watchdog

While ARMED, the controller must receive a valid communication heartbeat at least once every:

```
3 seconds
```

Instead of incrementing a 100-ms communication counter as you did in Lab 4, use elapsed time. Conceptually:

```
if((system_ms - last_comm_time) >= 3000)
{
    // Communication has been lost
}
```

The required response is:

```
ARMED → FAULT
```

and the ARMED LED must turn OFF. When entering ARMED, initialize the communication timer appropriately so the controller does not immediately generate a communication fault.

20. The Pushbutton Bounce Problem

Mechanical switches do not produce one perfect electrical transition. One physical press may look approximately like:

```
HIGH
 LOW
HIGH
 LOW
HIGH
 LOW
-----------------> time
```

With an external interrupt, these transitions may generate several interrupt requests. Do not solve this problem by writing:

```
__delay_ms(50);
```

inside an ISR. That would block the processor during the delay and defeat the purpose of the real-time architecture. Instead, implement a non-blocking debounce strategy using `system_ms`. A valid ARM or COMM event should not be accepted again until approximately:

```
50 ms
```

has passed since the previous accepted event. You must determine how to implement this using timestamps or event flags. The processor must continue operating during the debounce interval.

21. Status Heartbeat

Add a firmware heartbeat LED behavior using the 1-ms system clock. Use one of the existing state LEDs or an additional LED if available. The heartbeat timing should be:

State	Behavior
SAFE	Toggle every 500 ms
ARMING	Toggle every 200 ms

State	Behavior
ARMED	Toggle every 1000 ms
FAULT	OFF

Do not use `__delay_ms()` to generate these patterns. Use `system_ms` or independent software timing variables.

22. Required Firmware Architecture

Your finished program should conceptually follow this structure:

```
main()
 |
 +---- Configure oscillator
 |
 +---- Configure GPIO
 |
 +---- Configure pull-ups
 |
 +---- Configure INT0
 |
 +---- Configure INT1
 |
 +---- Configure INT2
 |
 +---- Configure interrupt priorities
 |
 +---- Configure Timer2
 |
 +---- Initialize state
 |
 +---- Enable interrupts
 |
 +---- while(1)
        |
        +---- Process fault_event
        |
        +---- Process arm_event
        |
        +---- Process comm_event
        |
        +---- Check ARM current level
        |
```

```
        +---- Check communication timeout
        |
        +---- Execute state machine
        |
        +---- Update heartbeat
        |
        +---- Update outputs
        |
        +---- Repeat
```

Meanwhile, the interrupt architecture should resemble:

```
HIGH-PRIORITY ISR
 |
 +---- INT0?
        |
        +---- Set fault_event
        +---- Clear INT0IF


LOW-PRIORITY ISR
 |
 +---- INT1?
 |      |
 |      +---- Set arm_event
 |      +---- Clear INT1IF
 |
 +---- INT2?
 |      |
 |      +---- Set comm_event
 |      +---- Clear INT2IF
 |
 +---- Timer2?
        |
        +---- Clear TMR2IF
        +---- system_ms++
```

You may organize Timer2 differently if your interrupt-priority configuration requires it, but you must be able to explain your design.

23. Important Design Challenge

Do not assume that every input should become an interrupt. You must use interrupts where an event needs asynchronous notification and polling where the current condition of a signal matters. For example:

```
FAULT occurs
```

```
        ↓
Interrupt

COMM packet arrives
        ↓
Interrupt

ARM button pressed
        ↓
Interrupt

Is ARM still being held?
        ↓
Read current pin level
```

A strong embedded design does not blindly replace every `if(PORT...)` statement with an interrupt.

24. ISR Rules

Your interrupt routines must follow these rules.

1. Do not use `__delay_ms()` inside an ISR.
2. Do not place the state machine inside an ISR.
3. Do not wait for another input inside an ISR.
4. Do not perform long loops inside an ISR.
5. Clear the correct interrupt flag.
6. Use `volatile` for variables shared between interrupt code and normal program code.
7. Keep high-priority interrupt processing especially short.
8. Do not make every interrupt high priority.

Your ISR should primarily detect, record, clear, and return.

25. What You Must Figure Out

Unlike an introductory interrupt exercise, you are not being given the complete program. You must determine how to configure INT0, INT1, and INT2; how to configure their falling-edge detection; how to configure interrupt priority mode; how to configure Timer2 for a 1-ms system clock; which variables must be `volatile`; how the ISR communicates with `main()`; how to implement the two-second ARM qualification without blocking; how to implement the three-second communication watchdog using elapsed time; how to reject switch bounce without delaying inside an ISR; how to guarantee that FAULT has the highest priority; and

when the current input level must still be polled. You already learned each individual concept in lecture. The challenge is combining them into one working embedded system.

26. Restrictions

Your final program must not use long blocking delays to implement the required behavior. The following solutions are not acceptable:

```
__delay_ms(2000);
__delay_ms(3000);
__delay_ms(500);
```

You may not poll the Timer2 interrupt flag from `main()` to create the system time.

You may not poll RB0 to detect the initial FAULT event.

You may not use software edge detection for the COMM button as you did in Lab 4.

You may not execute the complete state machine inside an ISR.

You may not place a debounce delay inside an ISR.

27. Required Demonstration

During the laboratory demonstration, demonstrate the complete system to the instructor.

1. Reset the PIC and demonstrate that the controller starts in SAFE.
2. Briefly press ARM for less than two seconds. Demonstrate that the system enters ARMING but returns to SAFE when ARM is released.
3. Hold ARM continuously for at least two seconds. Demonstrate that the controller enters ARMED.
4. Demonstrate that the system remains responsive during the two-second ARMING interval.
5. While ARMING, press FAULT. Demonstrate that the system immediately enters FAULT rather than waiting for the arming interval to finish.
6. Recover from FAULT by releasing both FAULT and ARM. Demonstrate that the system returns to SAFE and does not automatically return to ARMED.
7. Arm the controller again. Generate COMM events at intervals shorter than three seconds. Demonstrate that the system remains ARMED.
8. Stop generating COMM events. Demonstrate that approximately three seconds after the last valid communication event the controller enters FAULT.
9. Hold the COMM button continuously. Demonstrate that one physical press does not

behave like continuous communication. The communication watchdog must eventually expire.

10. Demonstrate the different heartbeat rates in SAFE, ARMING, and ARMED.
11. Explain which operations occur inside your ISR and which operations occur inside `main()`.
12. Explain why `fault_event`, `arm_event`, `comm_event`, and `system_ms` may need to be declared `volatile`.
13. Explain why ARM must still be read from the physical input during ARMING even though the initial ARM request uses an interrupt.
14. Explain what would happen if you forgot to clear an interrupt flag.
15. Explain why putting a 50-ms debounce delay inside an ISR would be poor firmware design.

28. What to Submit

Submit your complete MPLAB X project or source code as instructed by your instructor.

No formal laboratory report is required.

Your source code must be readable and appropriately commented. Comments should explain why important operations are performed rather than simply translating individual C statements into English.

Your program should clearly separate:

```
Hardware initialization

Interrupt service routines

Event processing

State-machine logic

Timing logic

Output control
```